

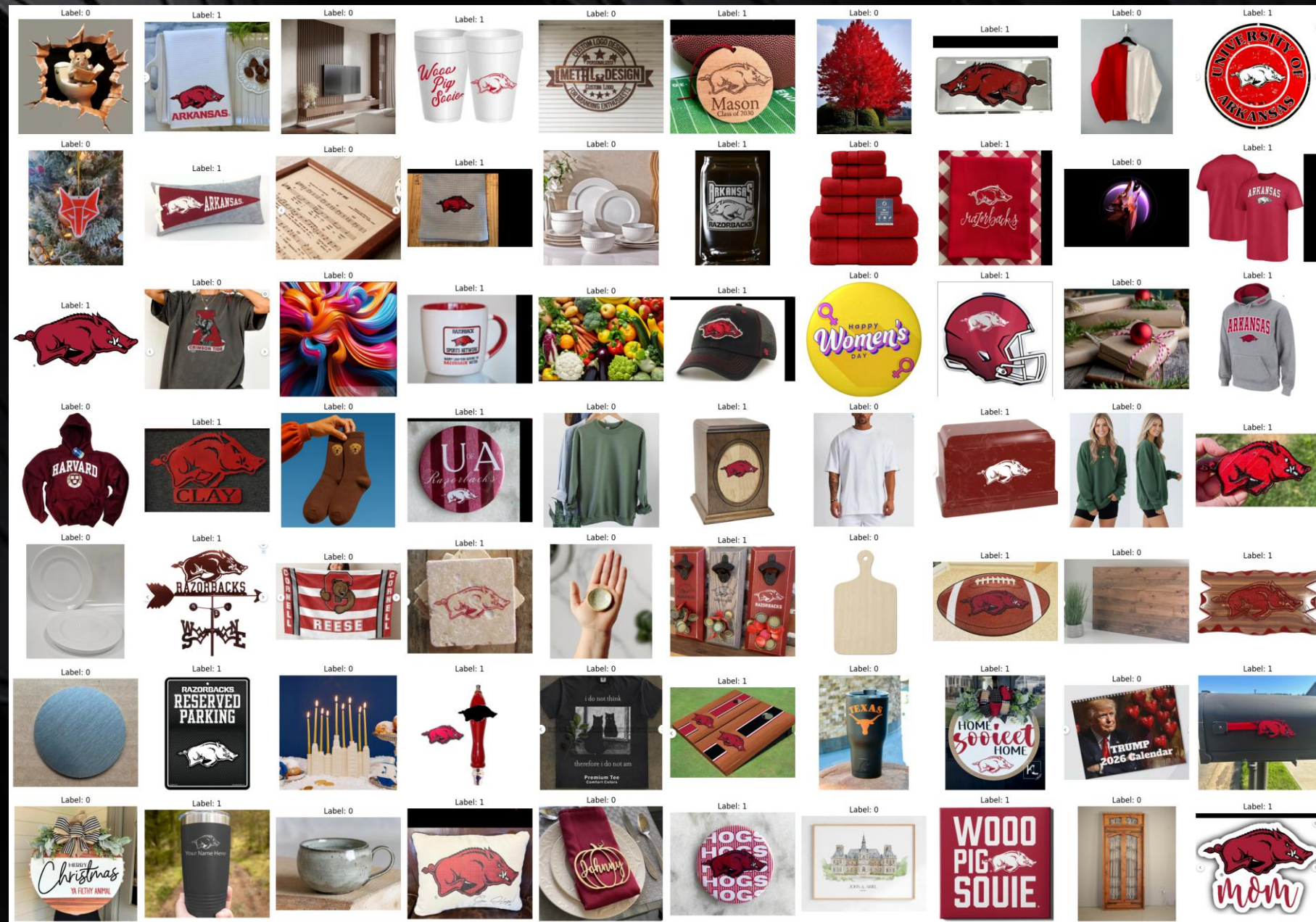
Applying CNN

Project 3-

Myli Brown and Ethan Stout

The Data

- Total of 100 images
- 50/50 class split:
 - 50 Razorbacks
 - 50 Non-Razorbacks



The Data: Transformations

- **Transformations for training and evaluation images:**
 - **Resize:** All images resized to 500×500
 - **Convert to Tensor:** Converts image to PyTorch tensor
 - **Normalize:** Standardizes pixel values using ImageNet mean and standard deviation
- **Goal**
 - Ensure consistent input size
 - Standardize pixel values for model stability
 - No random augmentation applied (so that the code was repeatable for class)

The Data: Split and Loading

- **Deterministic Split:**

- 70% Training
- 15% Validation
- 15% Testing
- Set (generator = g) to ensure that the splits are identical any time they are run.

- **DataLoaders:**

- Batch size = 8
- Training loader shuffles data but tied to the same (generator = g) to ensure repeatability.
- num_workers=0 to ensure a deterministic processing

- **Purpose:**

- Separate data ensures unbiased evaluation
- Fixed random seed (seed = 1) for full reproducibility
- Shared torch.Generator() controls all randomness
- Guarantees repeatable training, validation, and testing behavior

CNN: Model Architecture

- **Base Model:**
 - Pretrained ResNet18 (ImageNet weights)
- **Frozen Layers:**
 - conv1, bn1, layer1, layer2, layer3
- **Trainable Layers:**
 - Layer4 + final linear classifier
- **Classification Head:**
 - Single Linear Layer (no dropout)
 - Output = 1 (binary classification)
- **Purpose:**
 - Keep pretrained low-level features
 - Train only higher-level layers to adapt to Razorback / Not Razorback task
 - Ensures full determinism

CNN: Loss Function and Optimization

- **Loss Function:** BCEWithLogitsLoss
 - Combines Sigmoid + Binary Cross-Entropy
 - More numerically stable for small datasets
- **Optimizer:** Adam with weight decay
 - Learning rate = $1e-4$
 - Weight decay = 0.01 (regularization)
- **Purpose:**
 - Stable training on small dataset
 - Reduce overfitting via weight decay

CNN: Training and Validation Process

- **Epochs:**
 - Up to 30
 - Early stopping enabled if no improvement after 7 epochs.
- **Training Steps:**
 - Forward pass - compute logits
 - Compute loss - backpropagation
 - Optimizer step - update trainable parameters
 - Accumulate metrics - train loss & accuracy
- **Validation Steps:**
 - Forward pass only
 - Compute loss & accuracy
- **Batch Size:** 8

CNN: Results

- **Train Accuracy:**
 - Reaches 100% quickly because of the small size of the dataset
- **Validation Accuracy:**
 - Peaks around 80–87% meaning the model generalizes reasonably, but not perfectly.
- **Train Loss:**
 - Drops steadily meaning the model fits training data very well.
- **Validation Loss:**
 - Fluctuates, showing some overfitting and dataset sensitivity.
- **Early Stopping:**
 - Triggered at epoch 12 to help prevent further overfitting.

Epoch #	Training: Loss	Accuracy	Validation: Loss	Accuracy
Epoch 01	0.5728	71.43%	0.5343	73.33%
Epoch 02	0.1952	97.14%	0.5603	73.33%
Epoch 03	0.1104	98.57%	0.5410	80.00%
Epoch 04	0.1042	98.57%	0.6061	73.33%
Epoch 05	0.0654	98.57%	0.5080	86.67%
Epoch 06	0.1376	94.29%	0.5182	86.67%
Epoch 07	0.0429	100.00%	0.5449	73.33%
Epoch 08	0.0484	100.00%	0.5838	66.67%
Epoch 09	0.0411	100.00%	0.5422	66.67%
Epoch 10	0.0170	100.00%	0.5212	73.33%
Epoch 11	0.0077	100.00%	0.5231	80.00%
Epoch 12	0.0061	100.00%	0.5389	80.00%
Early stopping				

CNN: Results

Test Accuracy: 13/15 (86.67%)

✓ Pred: Razorback (93.1%)
True: Razorback



✗ Pred: Not Razorback (31.1%)
True: Razorback



✓ Pred: Not Razorback (28.7%)
True: Not Razorback



✓ Pred: Not Razorback (2.0%)
True: Not Razorback



✓ Pred: Not Razorback (12.9%)
True: Not Razorback



✓ Pred: Not Razorback (19.2%)
True: Not Razorback



✓ Pred: Razorback (99.9%)
True: Razorback



✓ Pred: Razorback (98.3%)
True: Razorback



✓ Pred: Razorback (81.9%)
True: Razorback



✓ Pred: Not Razorback (29.6%)
True: Not Razorback



✓ Pred: Razorback (92.7%)
True: Razorback



✗ Pred: Not Razorback (42.1%)
True: Razorback



✓ Pred: Razorback (99.7%)
True: Razorback



✓ Pred: Razorback (99.5%)
True: Razorback



✓ Pred: Razorback (97.5%)
True: Razorback



Reflection and Conceptual Questions:

How did you create your dataset and determine the split of official logo versus not official logo?

- Started with a dataset of 50 images with 25 of them being Razorbacks and 25 being not Razorbacks.
- Then, when the model performed poorly, we supplemented the images by adding 25 more images for each Razorback and not Razorback.
- We kept the class split to 50/50 because, after research, we determined that keeping the classes balanced was important in a small dataset so that the model would not be imbalanced toward one class.

Reflection and Conceptual Questions:

Why did you choose the specific architecture for the final model?

- **ResNet18 (Pretrained on ImageNet):**
 - Provides powerful pretrained feature extraction.
 - Suitable for small datasets because early layers already capture general visual features.
- **Frozen Early Layers:**
 - Layers 1–3 are frozen to retain the learned general features.
 - Reduces the number of parameters to train which prevents overfitting on a small dataset.
 - Preserves general features learned from ImageNet, so you don't "unlearn" them with limited Razorback data.
- **Trainable Later Layers + Custom Head:**
 - Layer 4 + linear output adapt the model to the specific task (binary classification).
- **Overall Strategy:**
 - Leverages transfer learning to get high performance with limited data.
 - Balances model complexity and overfitting risk.

Reflection and Conceptual Questions:

How did you monitor and mitigate overfitting?

- **Monitoring:**

- Tracked training vs. validation loss and accuracy each epoch.
- Observed gaps between training and validation performance to detect overfitting.

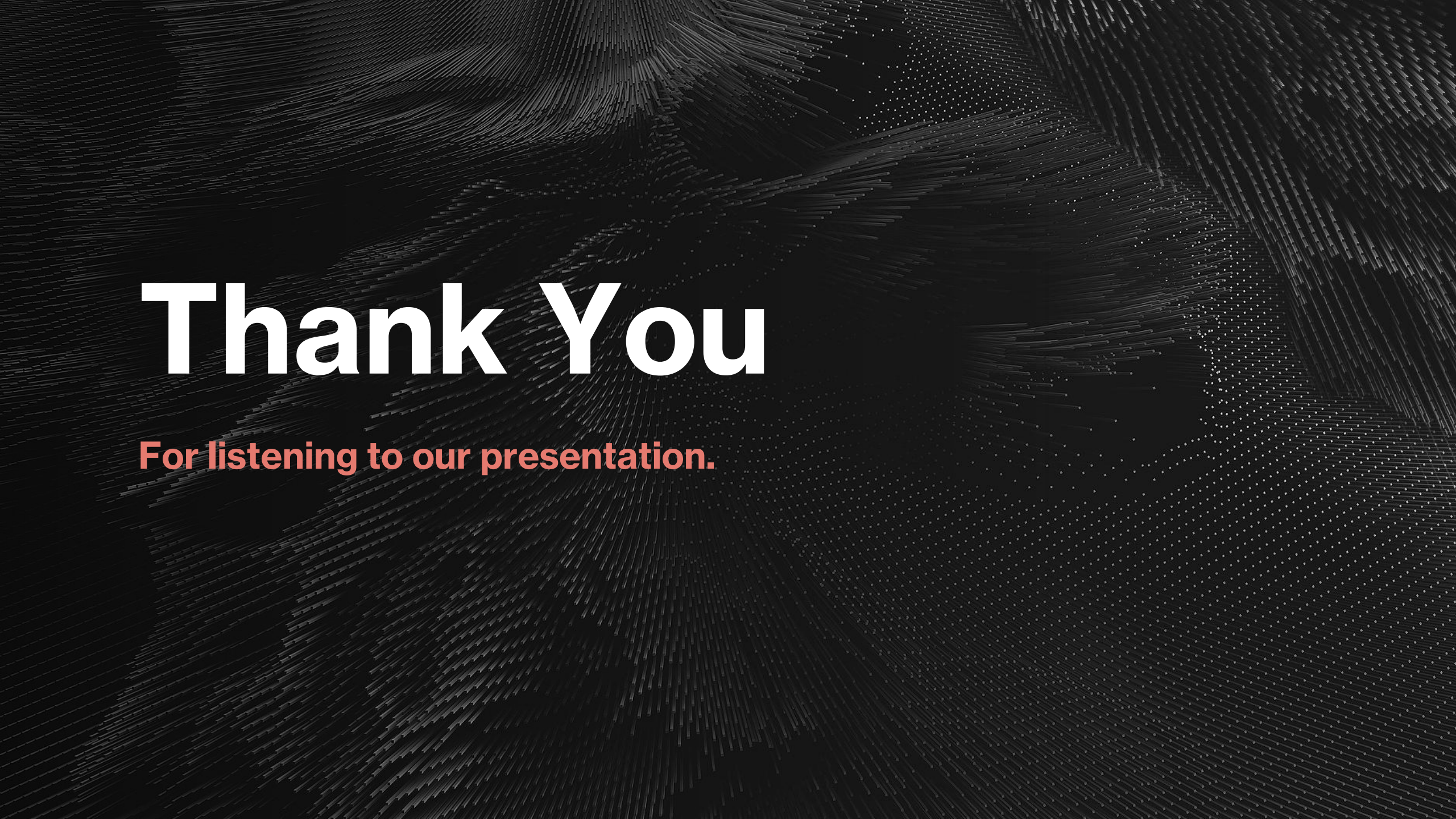
- **Mitigation Strategies:**

- **Early Stopping:** Stops training if validation loss does not improve for 7 epochs.
- **Frozen Layers:** Early ResNet layers frozen to reduce trainable parameters.
- **Weight Decay:** Added in Adam optimizer to penalize large weights.

Reflection and Conceptual Questions:

What future efforts do you recommend to improve model performance?

- **Increase Dataset Size:**
 - More images improve generalization.
- **Fine-Tune More Layers:**
 - Adapt pretrained features to our task.
- **Hyperparameter Optimization:**
 - Adjust learning rate, dropout, batch size, and weight decay.



Thank You

For listening to our presentation.